

NetStream: Observing Adaptive Bitrate Streaming Under Controlled Network Conditions

KSVKORD

April 2025

Contents

1	Introduction	3
2	Background Knowledge	3
2.1	How Video Streaming Works	3
2.2	Bitrate and Video Quality	3
2.3	Adaptive Bitrate Streaming and HLS	4
2.4	Types of Network Problems	4
2.5	tc netem: Simulating Network Conditions	4
3	Project Goal	5
4	System Implementation	5
4.1	Architecture Overview	5
4.2	Server	5
4.3	Browser Frontend	6
4.4	Network Simulation Script	6
5	Experiment Design	6
5.1	Test Video	6
5.2	Procedure	6
5.3	Collected Data	7
6	Results	7

7	Analysis	8
7.1	Why Latency Did Not Cause a Quality Drop	8
7.2	Why Packet Loss Did Not Cause a Quality Drop	9
7.3	Why Congestion Caused a Quality Drop	9
7.4	The Core Finding	9
8	Limitations	10
9	Conclusion	10

1 Introduction

Every time you watch a video on YouTube, Netflix, or any modern streaming platform, the quality of the video adjusts automatically based on your internet connection. When your connection is fast, you get crisp high-definition video. When it slows down, the video drops to a lower resolution to avoid buffering. You might have noticed this happening in real time.

This automatic adjustment is not magic. It is a system called Adaptive Bitrate Streaming, or ABR. This project builds a working video streaming server from scratch to understand exactly how ABR behaves when the network is put under different types of stress. By simulating real network problems in a controlled environment, we can observe and measure how the streaming system responds.

2 Background Knowledge

Before describing what was built and tested, it helps to understand a few basic ideas that the project builds on.

2.1 How Video Streaming Works

When you stream a video online, the video file is not sent to you all at once. Instead, it is cut into small pieces called segments, each a few seconds long. Your device downloads these segments one at a time and plays them back continuously, staying slightly ahead of what you are currently watching.

This approach has a key advantage. If one segment arrives slowly, your device can keep playing from the segments it has already downloaded while waiting. The amount of video stored and ready to play is called the buffer. A large buffer means the stream can survive network slowdowns without freezing.

2.2 Bitrate and Video Quality

Video quality is measured by bitrate, which is the amount of data per second needed to play the video. High resolution video requires a higher bitrate. For this project, two quality tiers were used.

Quality	Resolution	Required Bitrate
Low	360p	500 kbps
High	720p	2,500 kbps

If the network cannot deliver at least 2,500 kilobits per second, a player smart enough to do so will switch from 720p to 360p to keep the video playing smoothly.

2.3 Adaptive Bitrate Streaming and HLS

Adaptive Bitrate Streaming is the technology that allows a player to switch quality tiers automatically. The player constantly measures how fast it is downloading data and uses that measurement to decide which quality level to request next.

HTTP Live Streaming, or HLS, is the specific protocol used in this project. HLS works by encoding a video into multiple quality versions and breaking each version into small segment files. A special text file called a master playlist lists all available quality tiers. When the player opens a stream, it reads the master playlist, then chooses which tier to download based on the available bandwidth.

The player used in this project is hls.js, an open source JavaScript library that implements HLS in the browser. hls.js monitors download speeds and switches quality tiers automatically without any input from the user.

2.4 Types of Network Problems

Not all network problems are the same. This project tests three distinct types of degradation.

Latency is delay. When a packet is sent from a server, latency is the time it takes to arrive. High latency slows down the start of each segment download but does not reduce the total amount of data that can be delivered.

Packet loss happens when data packets are lost in transit and never arrive. The internet's main data transport protocol, TCP, automatically detects lost packets and re-sends them. This means the receiver eventually gets all the data, but the retransmissions take extra time and can create unpredictable delays.

Bandwidth cap is a hard limit on how much data per second can pass through a connection. Unlike latency or loss, a bandwidth cap directly reduces throughput. If a video tier requires more bandwidth than the cap allows, it is simply impossible to deliver that tier on time.

2.5 tc netem: Simulating Network Conditions

Testing a streaming server over a real degraded network is impractical in a lab setting. Instead, this project uses a Linux kernel tool called tc netem, which stands for Traffic Control Network Emulator. It operates at the operating system level and injects real delay, packet loss, or bandwidth

caps into a network interface. Because it works at the kernel level, the TCP protocol sees these conditions as genuine and responds exactly as it would on a real degraded connection.

3 Project Goal

The goal of this project is to build a complete HLS video streaming system and then use `tc netem` to apply four different network conditions to that system. By measuring how the streaming player responds to each condition, we can answer a concrete question: which types of network problems actually cause a streaming player to drop video quality, and which ones does it absorb without the viewer noticing?

4 System Implementation

4.1 Architecture Overview

The system has three parts: a server that processes and serves video, a browser frontend that plays the video, and a network simulation script that degrades the connection between them.

The server runs inside Ubuntu WSL2 on Windows 11. The browser is Windows Chrome, which connects to the server at `localhost:3000`. Network conditions are applied using `tc netem` on the loopback interface inside WSL2, which is the path that Chrome traffic travels through in this environment.

4.2 Server

The server is built with Node.js and Express. When a video file is uploaded, the server passes it to FFmpeg, which encodes two versions: a 360p version at 500 kbps and a 720p version at 2,500 kbps. Each version is split into 4-second segments and saved as HLS files. After encoding, the server writes a master playlist that lists both versions with their bandwidth requirements.

The server exposes three endpoints that the browser uses. The upload endpoint receives the video file. The status endpoint lets the browser check whether processing is complete. The videos endpoint lists all videos that are ready to play.

FFmpeg is run using the Node.js child process spawn function rather than `exec`. This distinction matters because `exec` buffers all output in memory, and a long video generates enough terminal output to crash the server. Spawn streams the output without storing it.

4.3 Browser Frontend

The browser interface has two pages. The first page lets the user upload a video and shows the processing status. Once a video is ready, a link appears in the library.

The second page is the player. It loads hls.js from a content delivery network and initialises playback using the master playlist. A statistics panel next to the video updates every second and shows six measurements: the current quality level, the estimated bandwidth, the buffer length in seconds, the total number of quality switches, the total number of buffer stalls, and the elapsed playback time.

The player page also includes an automatic recorder that captures a snapshot of all statistics every 30 seconds for three minutes and exports the results as a CSV file.

4.4 Network Simulation Script

The network simulation is controlled by a shell script called netem.sh. The script accepts a profile name as an argument and applies the corresponding tc netem rules to the loopback interface. It clears any existing rules before applying a new profile, so profiles can be switched cleanly between runs.

Profile	Delay	Packet Loss	Bandwidth Cap
Baseline	0 ms	0%	None
High Latency	150 ms	0%	None
Packet Loss	0 ms	3%	None
Congested	50 ms	1%	1.5 Mbps

5 Experiment Design

5.1 Test Video

A single gameplay video clip was used for all four runs to ensure consistency. The clip is shorter than three minutes, so the video was set to loop automatically during recording. This means the player continuously requests segments throughout the observation window.

5.2 Procedure

Each profile was tested using the same procedure. The server was restarted to clear any in-memory state. The player page was opened in Chrome with the browser cache disabled. The network pro-

file was applied using netem.sh. Playback was started and the automatic recorder was launched immediately. Statistics were captured every 30 seconds for a total of six samples over three minutes. After each run, the network was cleared and a 30-second pause was observed before the next profile.

The browser tab was kept active and visible throughout each run. Modern browsers throttle JavaScript timers in background tabs, which would corrupt the one-second update interval used by the statistics panel.

5.3 Collected Data

Six samples were collected per profile, giving 24 total data points.

Profile	t=30s	t=60s	t=90s	t=120s	t=150s	t=180s
<i>Quality level</i>						
Baseline	720p	720p	720p	720p	720p	720p
High Latency	720p	720p	720p	720p	720p	720p
Packet Loss	720p	720p	720p	720p	720p	720p
Congested	360p	360p	360p	360p	360p	360p
<i>Bandwidth estimate (kbps)</i>						
Baseline	369,257	369,257	369,257	369,257	369,257	369,257
High Latency	69,755	69,755	69,755	69,755	69,755	69,755
Packet Loss	207,238	207,238	207,238	207,238	207,238	207,238
Congested	1,501	1,501	1,501	1,501	1,501	1,501
<i>Buffer stalls</i>						
All profiles	0	0	0	0	0	0

6 Results

The left chart shows how each network profile affected the bandwidth estimate that hls.js measured. The scale is logarithmic because the values span a range of roughly 250 to 1, from 369,257 kbps under baseline to 1,501 kbps under congestion. Three profiles sit far above the 2,500 kbps threshold required for 720p. Only the congested profile falls below it.

The right chart shows the quality level that hls.js selected in response. Baseline, high latency, and packet loss all resulted in 720p throughout the observation period. The congested profile resulted in 360p from the very first sample and remained there for all six samples.

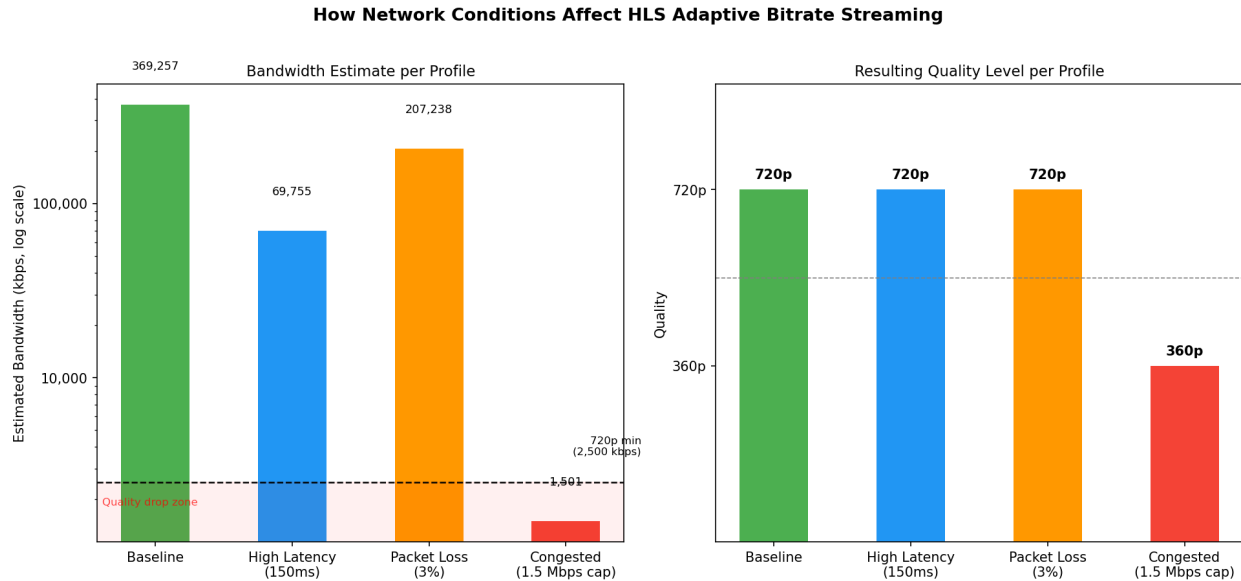


Figure 1: Left: average bandwidth estimate per profile on a logarithmic scale. The dashed line marks 2,500 kbps, the minimum required to sustain 720p. Right: the resulting quality level selected by hls.js for each profile.

7 Analysis

7.1 Why Latency Did Not Cause a Quality Drop

The high latency profile added 150 milliseconds of delay to every packet. This slowed the start of each segment download, which caused hls.js to measure a lower bandwidth than under baseline conditions. The measured bandwidth fell from 369,257 kbps to 69,755 kbps, a reduction of roughly 80 percent.

Despite this reduction, 69,755 kbps is still approximately 28 times higher than the 2,500 kbps threshold for 720p. From hls.js's perspective, there was no reason to switch quality. The delay made each segment take longer to begin arriving, but once the data started flowing, it arrived at full speed. The player's buffer remained healthy throughout, and no stalls were recorded.

This result demonstrates an important property of delay: it reduces perceived throughput as measured by segment timing, but it does not restrict the actual volume of data that can be delivered. A highly latent connection can still sustain high quality video as long as the buffer can absorb the slower segment starts.

7.2 Why Packet Loss Did Not Cause a Quality Drop

The packet loss profile dropped 3 percent of packets at the network level. TCP detected these losses and retransmitted the missing data automatically. The retransmissions added variable extra time to some segment downloads, which was reflected in a lower bandwidth estimate of 207,238 kbps compared to the baseline of 369,257 kbps.

As with the latency profile, 207,238 kbps is still far above the 2,500 kbps threshold. hls.js did not switch quality, and no buffer stalls occurred. The TCP retransmission mechanism absorbed the loss without the player noticing.

This result illustrates why TCP is described as a reliable transport protocol. The 3 percent loss was completely invisible to the application layer. A viewer watching this stream would not have experienced any degradation.

7.3 Why Congestion Caused a Quality Drop

The congested profile applied a hard bandwidth cap of 1.5 Mbps, or 1,500 kbps, along with 50 milliseconds of delay and 1 percent packet loss. The bandwidth cap is fundamentally different from the other conditions because it places a physical ceiling on throughput. No amount of retransmission or buffering can deliver data faster than the cap allows.

The 720p tier requires 2,500 kbps. With the cap set at 1,500 kbps, it is mathematically impossible to sustain 720p playback. hls.js measured a bandwidth of 1,501 kbps, recognized that this fell below the 720p threshold, and selected 360p instead. This decision was made before the first 30-second sample was recorded, and the quality remained at 360p for the entire three-minute run with no switches and no stalls.

This is the expected behavior of a well-designed ABR system. Rather than attempting 720p and running out of data, hls.js proactively selected a quality level that the network could actually sustain.

7.4 The Core Finding

The three degraded profiles produced three very different bandwidth estimates: 69,755, 207,238, and 1,501 kbps. Only one of them crossed the quality threshold. This demonstrates that what matters for video quality is not whether the network is degraded, but whether the degradation reduces throughput below what the current quality tier requires.

Latency and packet loss are forms of degradation that TCP can compensate for. They slow things

down but do not prevent data from arriving. A bandwidth cap cannot be compensated for. If the pipe is too narrow, the data simply cannot get through fast enough, and the ABR system must adapt by choosing a lower quality tier.

8 Limitations

This experiment was conducted on a single machine using loopback networking. All traffic between the browser and the server travelled within the same computer, which eliminates real-world factors such as routing variation, physical link instability, and congestion from other users sharing the same connection.

The controlled environment is a strength in one sense: results are reproducible and the variables are isolated. But it means the findings describe how the ABR algorithm responds to idealized network conditions, not to the noisy and unpredictable conditions of a real internet connection.

Additionally, only one video clip was tested. A longer clip with more varied scene complexity would produce more realistic segment sizes and potentially more variable bandwidth measurements.

9 Conclusion

This project built a working HLS video streaming server, applied four different network conditions using `tc netem`, and measured how the `hls.js` adaptive bitrate player responded to each one.

The key finding is that adaptive bitrate streaming is specifically sensitive to bandwidth constraints, not to network degradation in general. High latency and moderate packet loss, both common in real internet connections, were absorbed by TCP and the player's buffer without any change in video quality. Only a hard bandwidth cap that fell below the minimum requirement for the current quality tier caused the player to switch to a lower resolution.

This is the intended behavior of an adaptive bitrate system. It does not react to every form of network stress. It reacts to the one condition it cannot work around: not having enough bandwidth to sustain the current quality level.